

Amazon Reviews Analysis at Scale Using MapReduce

NASSALI VICTOR
2025/HD05/26362U

NAKAZZI SOLOME
2025/HD05/26358U

SSENONO HENRY
2025/HD05/26373U

June 5, 2026

Abstract

This report presents a large-scale data mining project utilizing the MapReduce programming model to analyze customer reviews and metadata from the Amazon Reviews 2023 Dataset. Focusing on the Software category subset, we designed and implemented five customized MapReduce jobs using Python's `mrjob` framework: Review Counting, Average Rating Computation, Top-Ten Product Identification, Helpful Vote Evaluation, and VADER-based Sentiment Analysis. The results demonstrate the scalability and analytical power of the MapReduce paradigm for processing unstructured consumer feedback, transforming raw transactional review streams into structured insights for business intelligence and recommendation engines. All the source files and content can be found at this public Github repository: <https://github.com/4Solome/Data-Mining-Project>

Contents

1	Introduction & Background	3
2	Dataset Analysis and Preprocessing	3
2.1	Data Schema	3
2.2	Preprocessing Pipeline	3
3	Tasks	4
3.1	Task 1: Counting reviews per product	4
3.2	Task 2: Computing average star rating per product	5
3.3	Task 3: Identifying top ten most reviewed products	5
3.4	Task 4: Evaluating Helpful vote ratios	6
3.5	Task 5: Text Sentiment Analysis	7
4	Analysis of Results	9
4.1	Product Ratings Distribution	9
4.2	Sentiment Classification Analysis	9
5	Conclusion	11

1 Introduction & Background

The exponential expansion of digital marketplaces has led to an explosion of consumer-generated textual data. Extracting meaningful patterns, ratings, and sentiments from millions of product reviews is critical for customer feedback loops, inventory strategy, and product recommendation engines. However, processing such massive volumes of data exceeds the operational limits of traditional sequential execution pipelines.

MapReduce is a standard, highly scalable framework for parallel processing across distributed clusters. By decomposing complex workflows into atomic **Map** and **Reduce** phases, MapReduce achieves massive scalability and fault tolerance. In this project, we utilize Python’s `mrjob` library to execute MapReduce pipelines locally, simulating a distributed ecosystem. This report outlines our methodology, codebase implementation, and exploratory data analysis (EDA) of the analyzed results.

All source code, scripts, notebooks, and result sets can be accessed directly at the project repository:

<https://github.com/4Solome/Data-Mining-Project>

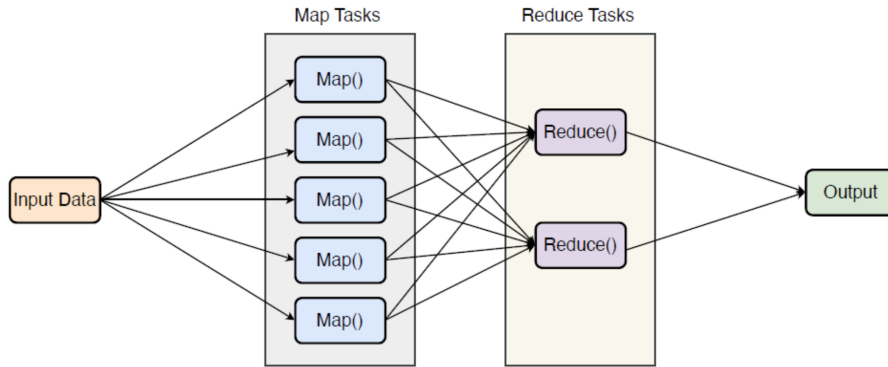


Figure 1: MapReduce overall architecture and execution pipeline.

2 Dataset Analysis and Preprocessing

We utilize the Amazon Reviews Dataset compiled in 2023 by McAuley Lab, which includes over 571 million reviews globally. To match local computational parameters, we isolated the **Software** category subset.

2.1 Data Schema

The raw dataset is stored in standard JSON-Lines (`.jsonl`) format. Each JSON record represents an individual review containing the schema detailed in Table 1.

2.2 Preprocessing Pipeline

A specialized Python dataset analysis tool (`dataset.py`) was built to read the raw files, count basic elements, handle null records, and a Jupyter notebook that would write the consolidated dataset into a clean runtime file at `data/software-clean.jsonl`. This step removed corrupt JSON items, ensuring downstream MapReduce code runs without interruption such as division by zero.

Table 1: Software Review Dataset Schema details.

Field	Type	Explanation
rating	float	Product score given by consumer (ranging from 1.0 to 5.0)
title	string	Textual headline summary of the review
text	string	Primary body paragraph of the review
asin	string	Unique Amazon Standard Identification Number (Product ID)
parent_asin	string	Parent identification number for grouping product variations
user_id	string	Unique identification key for the reviewer
timestamp	integer	Epoch Unix timestamp when the review was generated
verified_purchase	boolean	Verification indicator if purchase was completed on Amazon
helpful_vote	integer	Number of positive feedback votes identifying review quality

3 Tasks

In this section, we present the methodology, python scripts, and output patterns for each of our analytical pipelines.

3.1 Task 1: Counting reviews per product

To determine the frequency of user interactions per product, we mapped each record using the `asin` key. The mapper emits a pair of (`asin`, 1) for every valid review. The MapReduce shuffle-and-sort phase routes all occurrences of an `asin` to the same reducer, which performs a summation of values to deliver the total volume.

```

1 from mrjob.job import MRJob
2 import json
3
4 class MRReviewsCount(MRJob):
5
6     def mapper(self, _, line):
7         try:
8             review = json.loads(line)
9             asin = review.get('asin')
10            if asin:
11                yield asin, 1
12        except json.JSONDecodeError:
13            pass
14
15    def reducer(self, key, values):
16        yield key, sum(values)
17
```

```

18 if __name__ == "__main__":
19     MRReviewsCount.run()
20

```

Listing 1: MapReduce program to count reviews per product.

Output (clipped sample):

```

"B00004Y7D4" 24
"B000051S8K" 5
"B0000531AF" 112

```

3.2 Task 2: Computing average star rating per product

The rating distribution is calculated by capturing the real number ratings corresponding to each item. The mapper extracts the product ID (`asin`) and the corresponding numerical value `rating`, yielding (`asin`,`rating`). In the reducer, the aggregated values are cast to a list, aggregated via summation, and divided by the element count.

$$\text{Average Rating} = \frac{\sum_{i=1}^N \text{rating}_i}{N}$$

```

1 from mrjob.job import MRJob
2 import json
3
4 class MRAverageRating(MRJob):
5
6     def mapper(self, _, line):
7         try:
8             review = json.loads(line)
9             asin = review['asin']
10            rating = review['rating']
11            yield asin, rating
12
13        except:
14            pass
15
16    def reducer(self, key, values):
17        ratings = list(values)
18        total = sum(ratings)
19        count = len(ratings)
20
21        if count > 0:
22            yield key, total / count
23
24 if __name__ == "__main__":
25     MRAverageRating.run()
26

```

Listing 2: MapReduce job to calculate average star ratings.

3.3 Task 3: Identifying top ten most reviewed products

Finding a global Top-10 list requires a multi-step MapReduce program because a simple reducer only aggregates values locally per key. In Step 1, the mapper yields (asin, 1) and the reducer yields (None, (asin, $\sum 1$)). This structures all results under a single virtual key (None). In Step 2, a single reducer receives this combined list, sorts them in descending order by review count, and returns the first 10 items.

```
1 from mrjob.job import MRJob
2 import json
3 from mrjob.step import MRStep
4
5 class MRTop10Products(MRJob):
6
7     def mapper(self, _, line):
8         try:
9             review = json.loads(line)
10            asin = review.get('asin')
11            if asin:
12                yield asin, 1
13        except json.JSONDecodeError:
14            pass
15
16    def reducer(self, asin, values):
17        yield None, (asin, sum(values))
18
19    def reduceTopTen(self, _, values):
20        top = sorted(values, reverse=True)[:10]
21        for asin, count in top:
22            yield asin, count
23
24    def steps(self):
25        return [
26            MRStep(mapper=self.mapper, reducer=self.reducer),
27            MRStep(reducer=self.reduceTopTen)
28        ]
29
30 if __name__ == "__main__":
31     MRTop10Products.run()
32
```

Listing 3: Multi-step job to isolate top ten reviewed items.

3.4 Task 4: Evaluating Helpful vote ratios

We process helpfulness by classifying reviews into binary buckets. The mapper examines the `helpful_vote` field; if the value is 0, it yields ("unhelpful", 1), otherwise it yields ("helpful", 1). Using a secondary reduction step, the global ratio is evaluated:

$$\text{Average Helpfulness Score} = \frac{\text{Helpful Votes}}{\text{Helpful} + \text{Unhelpful Votes}}$$

```

1  from mrjob.job import MRJob
2  import json
3  from mrjob.step import MRStep
4
5  class MRHelpfulness(MRJob):
6
7      def mapper(self, _, line):
8          try:
9              review = json.loads(line)
10             vote = review.get('helpful_vote')
11             if vote == 0:
12                 yield 'unhelpful', 1
13             else:
14                 yield 'helpful', 1
15         except json.JSONDecodeError:
16             pass
17
18     def reducer(self, vote, values):
19         yield None, (vote, sum(values))
20
21     def reducerAvg(self, _, values):
22         helpful = 0
23         unhelpful = 0
24
25         for vote, count in values:
26             if vote == "helpful":
27                 helpful = count
28             else:
29                 unhelpful = count
30
31         total = helpful + unhelpful
32         avg = helpful / total if total > 0 else 0
33
34         yield "ave_helpful_score", avg
35
36     def steps(self):
37         return [
38             MRStep(mapper=self.mapper, reducer=self.reducer),
39             MRStep(reducer=self.reducerAvg)
40         ]
41
42     if __name__ == "__main__":
43         MRHelpfulness.run()
44

```

Listing 4: Evaluating customer helpfulness ratios using multi-step execution.

3.5 Task 5: Text Sentiment Analysis

To assess user satisfaction, we performed text-based Sentiment Analysis. In the mapper class, we initialized the NLTK `SentimentIntensityAnalyzer` (VADER). It parses raw text data and computes a compound metric score ranging between -1 and $+1$.

- **Positive Sentiment:** Score ≥ 0.05

- **Negative Sentiment:** $\text{Score} \leq -0.05$
- **Neutral Sentiment:** $-0.05 < \text{Score} < 0.05$

The reducer aggregates the frequency counters across the three distinct sentiments.

```

1  from mrjob.job import MRJob
2  from nltk.sentiment import SentimentIntensityAnalyzer
3  import json
4
5  class MRSentimentAnalysis(MRJob):
6
7      def mapper_init(self):
8          self.analyzer = SentimentIntensityAnalyzer()
9
10     def mapper(self, _, line):
11         try:
12             review = json.loads(line)
13             text = review.get("text", "")
14
15             if text:
16                 score = self.analyzer.polarity_scores(text)["
compound"]
17
18                 if score >= 0.05:
19                     yield "positive", 1
20                 elif score <= -0.05:
21                     yield "negative", 1
22                 else:
23                     yield "neutral", 1
24             except Exception:
25                 pass
26
27     def reducer(self, sentiment, counts):
28         yield sentiment, sum(counts)
29
30 if __name__ == "__main__":
31     MRSentimentAnalysis.run()
32

```

Listing 5: Distributed sentiment evaluation of review text using VADER NLP.

4 Analysis of Results

We collected the output values of our MapReduce processes, converted them to structured format (CSV), and evaluated them through a jupyter notebook using `matplotlib` and `pandas`.

4.1 Product Ratings Distribution

The rating system demonstrates extreme behavior. Most products gather scores concentrated closely around the extremes (1.0 and 5.0). This dual-peak outcome is typical in retail reviews: extremely happy customers write reviews, as do highly frustrated customers, whereas moderately satisfied buyers rarely write reviews.

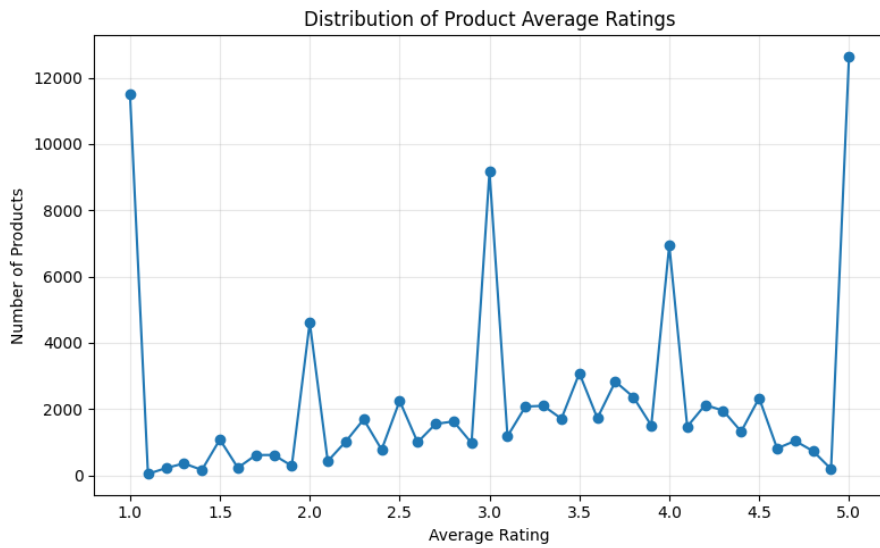


Figure 2: Empirical distribution of average product ratings demonstrating dual-peak clustering.

4.2 Sentiment Classification Analysis

We compiled the sentiment analysis classifications from all records giving:

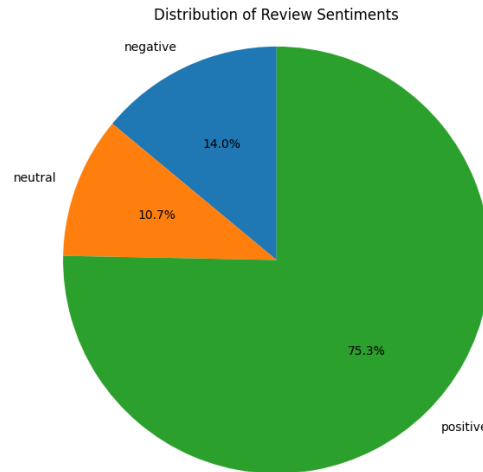


Figure 3: Review classifications into Positive, Neutral, and Negative segments.

The distribution of review sentiments across the dataset is categorized as follows:

- **Positive** = 75.3%
- **Negative** = 14.0%
- **Neutral** = 10.7%

This heavily right-skewed positive presence indicates that users are overall highly satisfied with software products, or that positive language is naturally dominant in verified customer experiences.

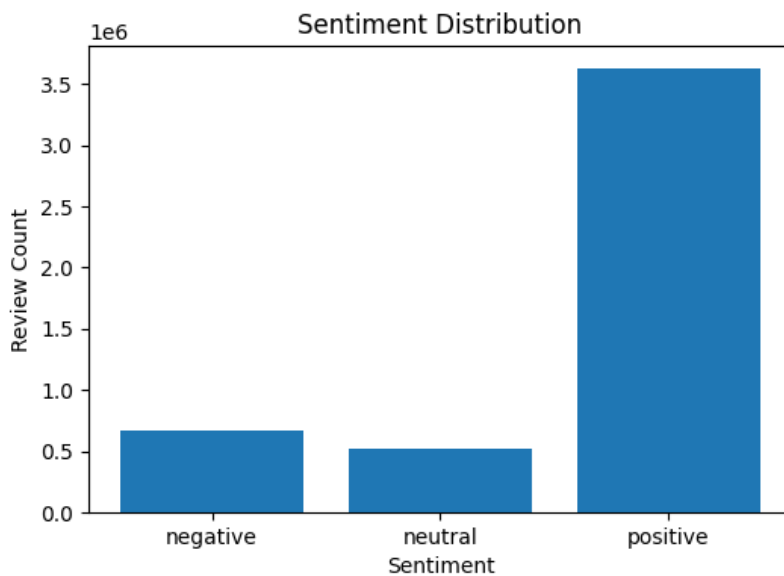


Figure 4: Absolute volume count of sentiment records across the software category.

5 Conclusion

This project successfully implemented a series of distributed MapReduce analytical tasks to query, aggregate, and process customer reviews in the Amazon Software category. By leveraging the `mrjob` framework, we demonstrated how high-volume sentiment classification, ranking, and statistical calculations can be split into local parallel execution units.

Future additions to this workflow could include:

1. **Deployment to AWS EMR:** Porting this localized implementation directly onto a live Hadoop/Spark architecture in Amazon Elastic MapReduce to process the complete 571-million review archive.
2. **Advanced NLP Modeling:** Upgrading basic VADER lexical rules to heavier contextual embedding classifiers, such as DistilBERT, during Map phases.
3. **Item-to-Item Collaborative Filtering:** Expanding basic metric counts into multi-pass jobs calculating co-occurrence vectors to build an active recommendation service.

References

1. McAuley, J., et al. *Amazon Reviews 2023 Dataset*. <https://amazon-reviews-2023.github.io/>
2. Dean, J., & Ghemawat, S. (2008). *MapReduce: Simplified Data Processing on Large Clusters*. ACM Communications. <https://static.googleusercontent.com/media/research.google.com/es/us/archive/mapreduce-osdi04.pdf>
3. Python `mrjob` Library Documentation. <https://mrjob.readthedocs.io/>
4. NLTK Sentiment Intensity Analyzer Documentation. <https://www.nltk.org/api/nltk.sentiment.SentimentIntensityAnalyzer.html>